

Capítulo 6

1._ E

Aquí se quiere que `numberBags` termine valiendo 2, pero esta dentro del constructor sin parámetros. Entonces, si en la línea 1 pones `this(2);`, estás llamando al otro constructor (`BirdSeed(int numberBags)`), que sí asigna el valor con `this.numberBags = numberBags;`. Por eso ahora sí imprime 2

Las demás:

A y B están mal porque intentan llamar al constructor sin `new`, y eso no se puede.

C y D sí crean un objeto con `new BirdSeed(2)`, pero ese objeto es otro, no el actual, entonces `seed.numberBags` sigue en 0.

F está mal porque no puedes usar `this(2);` en la línea 2, esa llamada a otro constructor tiene que ser la primera línea.

G está mal porque sin cambios imprime 0, no 2.

2._ A, B y F

El `static` con `final` sí se puede (A), no hay conflicto. `private` con `static` también (B), totalmente válido. Y `private` con `final` (F) también se puede, aunque es medio redundante porque un método `private` ya no se puede sobrescribir, pero Java lo permite.

Las incorrectas:

C (`static + abstract`) no se puede porque `abstract` necesita que el método se sobrescriba, y `static` no se puede sobrescribir.

D (`private + abstract`) tampoco, porque un método `abstract` debe ser visible para que otra clase lo implemente, y `private` lo bloquea.

E (`abstract + final`) es contradictorio: uno dice “sobrescríbeme” y el otro “no me puedes sobrescribir”.

3._ B y C

B es correcta porque los métodos `overridden` (los que sobrescribes en herencia) deben tener la misma firma (nombre + parámetros). C también es correcta porque los métodos `hidden` (los `static`) igual deben tener la misma firma.

Las demás:

A está mal porque los `overloaded` sí cambian la firma (parámetros), no la mantienen igual. En la D está mal porque los `overloaded` pueden tener distinto tipo de retorno. En la E está mal porque los `overridden` pueden cambiar el tipo de retorno, no tiene que ser exactamente igual, y la F está mal por lo mismo, los `hidden` tampoco están obligados a tener el mismo `return`.

4._ F

El problema es que `Mammal` no tiene constructor vacío, solo tiene uno con parámetro (`int age`). Entonces cuando `Platypus()` se ejecuta, Java intenta meter un `super()` automático, pero como no existe, no compila. Por eso tienes que cambiar la línea 9 y llamar explícitamente a `super(2)` (o cualquier `int`), y así ya funciona.

La E está mal porque la línea 7 no tiene problema. El sneeze() de Mammal es private, entonces ni siquiera se hereda, o sea que el de Platypus no está sobrescribiendo nada y puede tener otro tipo de retorno sin bronca. Y las demás (A, B, C, D) están mal porque el código ni compila, entonces no puede haber salida.

5._ E

La correcta es la E. Aquí el detalle es que hay dos numSpots, uno en Speedster y otro en Cheetah. No se sobrescriben, se ocultan, entonces dependen del tipo de referencia. Como en main usas Speedster s, se va a usar el numSpots de Speedster, no el de Cheetah.

Entonces necesitas asignar el valor al de la clase padre, por eso E imprime 50.

Las demás:

A no hace nada útil, se asigna a sí mismo. en la B está al revés, no cambia nada.

C asigna el de Cheetah, pero ese no se usa en el print, en la D igual no sirve, solo lee el valor (0) y lo vuelve a asignar, en la F está mal porque sí compila y G está mal porque sí hay una correcta.

6._ D y E

Las correctas son D y E, ya que para que una clase sea immutable, básicamente debe ser final y sus atributos private final (o que no se puedan cambiar).

D (Elk) es correcta porque es final y no tiene nada que modificar, entonces es immutable. E (Deer) también, porque es final y su atributo es private final, entonces no puede cambiar después de crearse, en las demás:

A (Moose) está mal porque ni compila: el final int antlers no está inicializado.

B (Caribou) está mal porque no es final, entonces alguien puede heredar y hacerla mutable.

C (Reindeer) igual, aunque tiene final, la clase no es final, entonces no es realmente immutable, y la F está mal porque sí hay respuestas correctas

7._ A

La correcta es la A.

El código sí compila. Aquí hay sobrecarga y sobreescritura. El método printName(int) de Arthropod sí se sobrescribe en Spider, porque tiene misma firma y mayor visibilidad (protected).

Entonces:

En a.printName((short)4), el short se convierte a int, y como ese método está sobrescrito, se usa el de Spider y por ende imprime Spider.

En a.printName(4), igual es int, entonces otra vez se usa el de Spider que da Spider.

En a.printName(5L), ahora es long, entonces no entra al método sobrescrito, sino al de Arthropod que recibe long, lo que da Arthropod.

Las demás:

B y C y D están mal porque cambian qué método se ejecuta.

E y F están mal porque el código sí compila sin problema.

8._ D

La correcta es la D.

Primero, cuando creas `new Pelican()`, Java mete automáticamente un `super()`, entonces primero se ejecuta el constructor de Bird e imprime Wow-. Luego regresa al constructor de Pelican e imprime Oh-.

Después, cuando llamas `chirp.fly()`, se usa el método de Pelican, porque el de Bird es `private` (no se hereda ni se puede sobrescribir realmente). Entonces se ejecuta el de Pelican.

Las demás:

A y B están mal porque les falta el Wow-.

C está mal porque llamaría al método de Bird, pero ese es `private`.

E está mal porque el código sí compila sin problema.

9._ B y E

La respuesta correcta es B y E. Un método sobrescrito debe tener exactamente la misma firma, así que no pueden cambiar los parámetros, por eso la A es incorrecta. Sí puede lanzar nuevas excepciones siempre que no sean `checked`, entonces B es correcta. No es obligatorio que tenga más acceso, puede quedarse igual, así que C es falsa. Tampoco puede lanzar excepciones `checked` más amplias que las del método padre, por eso D es incorrecta. Y si el método original regresa `void`, el sobrescrito también debe ser `void`, así que E es correcta.

10._ A y C

La respuesta correcta es A y C. En el constructor de Howler, usar `this(3)` funciona porque llama al constructor que recibe `int`. También `this((short)1)` funciona porque `short` se convierte a `int`. En la clase Wolf, `this("")` y `this(null)` funcionan porque llaman al constructor que recibe `String`.

Las otras opciones fallan porque intentan llamar constructores que no existen o usan `super()` cuando la clase padre no tiene constructor sin parámetros.

11._ C

La respuesta es C. El código sí compila y corre bien. Primero se crea un objeto en la línea 16, donde `value` empieza como `"t"` y los bloques inicializadores le agregan `"a"` y `"c"`, quedando `"tac"`. Luego el constructor privado agrega `"b"`, quedando `"tacb"`, pero ese objeto se reemplaza después.

En la línea 17 se crea otro objeto con `"f"`. Otra vez inicia como `"tac"`, luego el constructor con `String` llama a `this()`, que agrega `"b"` (queda `"tacb"`), y después agrega `"f"`, quedando `"tacbf"`.

12._ C

La respuesta es C. Hay errores en 2 líneas. Primero, en la línea 8 hay error porque Beaver extiende de Rodent, pero Rodent no tiene constructor vacío. Entonces Beaver debería llamar explícitamente a super(Integer) en un constructor, y como no lo hace, falla.

Luego en la línea 9 hay dos errores: uno porque intenta cambiar el tipo de retorno de Integer a Number, pero eso no es covariante (es al revés), y otro porque el método en la clase padre es static y aquí no lo es, entonces no se puede sobrescribir así.

13._ A y G

La respuesta es A y G. El compilador solo crea un constructor por defecto cuando la clase no tiene ningún constructor definido.

En A no hay constructores, entonces sí se genera automáticamente uno vacío. En G tampoco hay constructor, porque Bird bird() en realidad es un método (tiene tipo de retorno), no un constructor, así que también se genera el constructor por defecto.

En las demás no aplica porque B y C ni siquiera compilan porque el nombre no coincide con la clase. D, E y F sí compilan, pero como ya tienen constructor, el compilador no agrega otro.

14._ B, E y F

La respuesta es B, E y F.

Una clase sí puede implementar muchas interfaces, por eso B es correcta. También, si una clase implementa una interfaz, automáticamente es un subtipo de esa interfaz, entonces E también es correcta. Y F describe correctamente qué es la herencia múltiple, aunque en Java no se permite con clases.

Las demás están mal porque A es falsa, solo puedes extender una clase, en la C es falsa porque los tipos primitivos no heredan de Object y D está al revés, B sería subclase de A, no superclase.

15._ C

El programa no compila porque en la clase abstracta Nocturnal se declara el método isBlind() sin cuerpo pero tampoco se marca como abstracto, lo cual no es válido en Java. Los métodos sin implementación deben declararse como abstractos dentro de una clase abstracta. Las demás líneas son correctas, incluyendo la sobrescritura del método en Owl y el uso del polimorfismo en el main.

16._ D

El programa compila correctamente. Primero se ejecutan los bloques static de las clases en orden de jerarquía, iniciando con Arachnid y luego Scorpion, dando como resultado "uq". Posteriormente, se imprimen dos veces antes de crear objetos. Al instanciar Arachnid se ejecutan sus bloques de instancia agregando "cr". Al instanciar Scorpion se ejecutan

primero los bloques de instancia de Arachnid y luego los de Scorpion, agregando "crm". Finalmente el resultado impreso es "uq uq uqcrcrm", por lo que la respuesta correcta es D.

17._ C y F

La opción C es correcta porque `this.variableName` puede utilizarse en cualquier método de instancia ya que hace referencia al objeto actual. La opción F también es correcta porque un método `main` dentro de la misma clase puede acceder a miembros privados, incluyendo constructores privados. Las opciones A y B son incorrectas porque `this()` solo puede usarse como primera línea dentro de un constructor. La opción D es incorrecta porque los métodos `static` no tienen acceso a `this`. La opción E es incorrecta porque el constructor por defecto del compilador no puede ser invocado explícitamente con `this()`.

18._ D y F

La opción D es correcta porque el método `static drink()` en `Monkey` no sobrescribe el método de `Mammal`, sino que lo oculta, ya que los métodos `static` no se pueden `override` sino que se aplican `method hiding`. La opción F es correcta porque el método `dance()` en `Monkey` tiene la misma cantidad de parámetros pero diferentes tipos, lo que lo convierte en una sobrecarga válida. Las demás opciones son incorrectas porque el método `eat()` de `Mammal` es `private` y no se hereda, por lo que no puede ser ni sobrescrito ni sobrecargado desde las subclases, y además la versión en `Monkey` no es válida como `override`.

19._ F

El código no compila porque la clase `Reptile` no define un constructor sin argumentos, por lo que el constructor de `Lizard` debe invocar explícitamente `super(int)`. Como esto no ocurre, el compilador intenta insertar `super()` automáticamente, pero falla porque no existe en la clase padre. Por esta razón, el error ocurre en la construcción del objeto y la respuesta correcta es F.

20._ E

El programa compila y ejecuta correctamente. Aunque la referencia es de tipo `Bird`, el objeto real es `Macaw`, por lo que se ejecuta el método sobrescrito `fly()` de `Macaw` debido al polimorfismo. Este método devuelve un nuevo objeto `Macaw` con valor `feathers` igual a 3. Luego se realiza un casting válido a `Parrot` ya que `Macaw` es subtipo de `Parrot`, y finalmente se accede al atributo `feathers` definido en `Bird`, por lo que el resultado impreso es 3.

21._ B y G

Una clase inmutable es aquella cuyo estado no puede cambiar después de ser creada. La opción B es correcta porque una clase inmutable no debe poder ser extendida, lo que se logra marcándola como `final`. La opción G también es correcta porque puede permitir acceso de solo lectura a datos mutables siempre que no exista forma de modificarlos. Las demás opciones son incorrectas porque una clase inmutable puede tener variables de instancia y estáticas, no requiere ser `static`, no requiere constructores privados obligatoriamente y no debe incluir `setters`.

22._ D

El programa compila y ejecuta correctamente. Existen dos variables estáticas separadas llamadas `name`, una en `Person` y otra en `Child`, lo que se conoce como `variable hiding`. Las asignaciones a través de `m` afectan a `Child.name`, mientras que las asignaciones a través

de t afectan a Person.name. El método setName() es sobrescrito, por lo que ambas referencias ejecutan la versión de Child, modificando únicamente Child.name. Al final, Child.name contiene "Olivia" y Person.name contiene "Sophia", por lo que la salida es "Olivia Sophia".

23._ B

El programa compila y ejecuta correctamente. Al crear un objeto Fennec, se encadenan los constructores desde la clase más derivada hacia la clase base. El constructor Fennec(int) llama a Fox(String), el cual llama a Fox(long), y este invoca implícitamente el constructor sin argumentos de Canine, agregando "q". Luego se ejecuta Fox(long) agregando "p", después Fox(String) agrega "z", y finalmente Fennec agrega "j". El resultado final es "qpzj".

24._ C

El programa compila y ejecuta correctamente. Primero se ejecutan los bloques static en orden de jerarquía, comenzando con Antelope y luego Gazelle, imprimiendo 1 y 8. Al crear el objeto Gazelle, se ejecutan primero los inicializadores de instancia de la clase padre Antelope, imprimiendo 2, seguido del constructor de Antelope que imprime 4. Luego se ejecutan los inicializadores de instancia de Gazelle, imprimiendo 9, y finalmente su constructor imprime 3. El resultado final es 182493.

25._ B y C

Una clase concreta es una clase no abstracta que puede ser instanciada. La opción B es correcta porque una clase concreta debe implementar todos los métodos abstractos heredados, de lo contrario debe declararse abstracta. La opción C también es correcta porque una clase concreta puede ser marcada como final para evitar herencia. Las demás opciones son incorrectas porque una clase concreta no puede ser abstracta, no necesita ser inmutable y los métodos sobrescritos no requieren una coincidencia exacta en todos los aspectos debido a reglas como covariant return types.

26._ D

El código no compila porque la referencia Whale no tiene definido un método dive que acepte parámetros. Aunque la clase Orca implementa un método varargs dive(int... depth), este no es visible desde la referencia de tipo Whale en el momento de compilación. Por lo tanto, la llamada whale.dive(3) es inválida y el error ocurre en la línea 8.

Capítulo 7

1._ B y D

Los records en Java son clases especiales implícitamente finales e inmutables. La opción B es correcta porque los records pueden declararse sin campos adicionales y son final por defecto. La opción D es correcta porque es válido sobrescribir los métodos generados automáticamente, como los accesores. La opción A es incorrecta por conflicto de nombres entre el campo del record y un campo estático. La opción C es incorrecta porque los records no pueden ser abstractos. La opción E es incorrecta porque los records son inmutables y no permiten métodos que modifiquen sus campos.

2._ A, B, D y E

El código compila correctamente. La variable puede declararse como Frog, TurtleFrog o CanHop debido a la relación de herencia e implementación de interfaces, ya que TurtleFrog es subtipo de Frog y Frog implementa CanHop. También es válido usar var, ya que el tipo se infiere como TurtleFrog. Las opciones incorrectas fallan porque BrazilianHornedFrog no es supertipo de TurtleFrog y Long no tiene relación con la jerarquía.

3._ C

El programa no compila porque el enum contiene miembros adicionales después de la lista de constantes, lo que requiere un punto y coma después del último valor del enum. Como no se incluye el punto y coma después de STRAWBERRY, el código genera un error de compilación en la declaración del enum. Por lo tanto, la respuesta correcta es C.

4._ C

El programa no compila porque la clase Armadillo extiende una clase sealed, por lo que está obligada a declararse como final, sealed o non-sealed. Al no incluir ninguno de estos modificadores, el código viola las reglas de sealed classes de Java y genera un error de compilación. Por esta razón, la respuesta correcta es C.

5._ E

El programa no compila porque la clase Beetle no implementa correctamente todos los métodos abstractos heredados. Aunque implementa getNumberOfLegs(), no implementa getNumberOfSections() con la firma exacta requerida, sino una versión sobrecargada con parámetros adicionales, lo cual no satisface el contrato de la interfaz HasExoskeleton. Por esta razón, la clase Beetle no es válida y la respuesta correcta es E.

6._ D y E

El código no compila porque el método chew() en la interfaz está declarado como abstracto pero tiene cuerpo, lo cual no está permitido. Además, la clase IsAPlant intenta extender una interfaz en lugar de implementarla, lo cual es incorrecto en Java. Por estas razones, las opciones D y E son correctas.

7._ E

El programa no compila porque el método getNumOfGills(int) en la clase ClownFish implementa un método de una interfaz que es implícitamente público, pero la implementación no declara el método como public, reduciendo su visibilidad. En Java no se

permite disminuir la visibilidad al sobrescribir métodos. Por esta razón, la línea 6 causa un error de compilación y la respuesta correcta es E.

8._ A,B y C

La encapsulación en Java requiere que los atributos de instancia sean privados para evitar acceso directo desde fuera de la clase. Los métodos pueden tener distintos niveles de acceso como public, protected o private dependiendo del diseño. Por lo tanto, las opciones A, B y C son correctas porque todas mantienen la variable encapsulada correctamente, mientras que la opción D es incorrecta porque expone directamente el atributo.

9._ A, E y F

El método setSnake solo acepta objetos del tipo Snake o de sus subclases. Cobra y GardenSnake son válidas porque pertenecen a la jerarquía de Snake. Snake no puede ser instanciada por ser abstracta. Object y String no pertenecen a la jerarquía, por lo que no son válidos. Además, null siempre es una asignación válida para referencias de objetos. Por lo tanto, las respuestas correctas son A, E y F.

10._ A, B, C, y E

El método privado en la interfaz Walk no se hereda, por lo que la clase Leopard no tiene restricciones en su implementación y puede retornar cualquier tipo compatible, como List, ArrayList o incluso tipos no relacionados con la interfaz. En cambio, la interfaz Run sí define un método público que debe ser implementado exactamente o con un tipo de retorno covariante, por lo que la clase Panther debe retornar ArrayList o un subtipo. Por lo tanto, las respuestas correctas son B, C y E.

11._ B

El programa compila correctamente. La clase interna Popcorn define una variable estática butter que oculta la variable de la clase externa Movie. En el método startMovie(), la referencia a butter sin prefijo hace referencia a la variable de la clase interna, por lo que se imprime el valor 10. Por lo tanto, la respuesta correcta es B.

12._ A, B y E

La encapsulación en Java permite proteger los datos de una clase haciendo que las variables de instancia sean privadas y proporcionando métodos públicos como getters y setters para acceder o modificarlas de forma controlada. No requiere convenciones de nombres específicas, y no permite que las variables sean públicas. Por lo tanto, las respuestas correctas son A, B y E.

13._ F

El programa no compila porque en un switch con enums no se deben usar los valores completamente calificados como Seasons.SPRING. En su lugar, deben usarse directamente los valores del enum como SPRING, WINTER y SUMMER. Debido a esto, múltiples líneas del switch contienen errores de compilación, por lo que la respuesta correcta es F.

14._ A, C y E

Las clases y interfaces sealed en Java restringen qué tipos pueden extenderlas o implementarlas mediante la cláusula permits. Una clase sealed puede ser extendida por clases finales, abstractas, sealed o non-sealed siempre que estén permitidas. También las

interfaces sealed restringen qué subinterfaces o clases pueden usarlas. No impiden herencia indirecta si se usa non-sealed. Por lo tanto, las respuestas correctas son A, C y E.

15._ G

El código no compila porque la clase Spirit está declarada como final, lo que impide que sea extendida. Sin embargo, en el método main se intenta crear una clase anónima que hereda de Spirit, lo cual no está permitido en Java. Por esta razón, ninguna de las opciones de llamada a métodos es relevante y la respuesta correcta es G.

16._ E

El código no compila porque la clase estática anidada OstrichWrangler intenta acceder a la variable de instancia count de la clase externa. Las clases estáticas no tienen acceso directo a miembros de instancia de la clase contenedora, lo que provoca un error de compilación en la línea 5. Por lo tanto, la respuesta correcta es E.

17._ E y G

El código no compila porque en las interfaces los métodos con cuerpo deben ser default o private. La línea 5 no especifica ninguno de estos modificadores. Además, las interfaces no permiten métodos protegidos, por lo que la línea 7 también es inválida. Las demás líneas son válidas debido a las reglas implícitas de interfaces en Java. Por lo tanto, las respuestas correctas son E y G.

18._ E

El programa no compila porque la clase Diet es una clase interna no estática, lo que requiere una instancia de la clase externa Deer para poder instanciarla. Sin embargo, el método main es estático y no tiene acceso a una instancia de Deer, por lo que new Diet() no es válido. Por esta razón, el código no compila en el main y la respuesta correcta es E.

19._ G

El código no compila porque el enum FOOD declara un método abstracto isHealthy(), lo que obliga a que todas las constantes del enum lo implementen. Sin embargo, solo INSECTS lo implementa, mientras que las demás constantes no lo hacen, lo cual viola la regla de enums con métodos abstractos. Por lo tanto, el programa no compila y la respuesta correcta es G.

20._ A, D y F

En Java, el overriding se resuelve en tiempo de ejecución mediante polimorfismo dinámico, por lo que el método ejecutado depende del tipo real del objeto. En cambio, los métodos estáticos utilizan method hiding y se resuelven en tiempo de compilación, dependiendo del tipo de referencia. Además, un método marcado como final no puede ser sobrescrito ni ocultado. Por lo tanto, las respuestas correctas son A, D y F.

21._ F

El código no compila porque el constructor definido no es compacto, por lo que Java exige que la primera línea sea una llamada a this(...) para inicializar los atributos del record, que son finales. Ninguna opción cumple con esto, por lo tanto el código no compila y la respuesta correcta es F.

22._ C, D y G

Las clases internas no estáticas requieren una instancia de la clase externa para ser creadas, por lo que Cub debe instanciarse usando `new Lion().new Cub()`. En cambio, la clase Den es estática, por lo que puede instanciarse directamente sin una instancia de Lion. Por ello, las respuestas correctas son C, D y G.

23._ D

Cuando una clase implementa múltiples interfaces con métodos default que tienen la misma firma, debe sobrescribir el método para resolver el conflicto. Sin embargo, puede invocar un método específico de una interfaz usando la sintaxis `Interfaz.super.metodo()`. Por lo tanto, `Swim.super.perform()` permite imprimir "Swim!", haciendo correcta la opción D.

24._ B y E

Las líneas 3 y 6 no compilan porque los métodos estáticos no pueden llamar directamente a métodos de instancia. En ambos casos se intenta invocar un método no estático desde un contexto estático, lo cual no es válido en Java. Por lo tanto, las respuestas correctas son B y E.

25._ B

El programa compila y ejecuta correctamente. Aunque la clase interna Stripes define su propio atributo x, este no se utiliza en la salida. En su lugar, se accede al atributo de la clase externa mediante `Zebra.this.x`, que tiene el valor 24. Por lo tanto, la salida es "x is 24", siendo correcta la opción B.

26._ C y F

El código no compila porque en un enum el constructor es implícitamente privado y no puede declararse como público. Además, cuando el enum contiene campos o métodos, la lista de valores debe terminar con punto y coma, lo cual falta en este caso. Por lo tanto, las respuestas correctas son C y F.

27._ B, C, D y G

El compilador genera automáticamente métodos de acceso para cada campo del record, además de implementar `toString()`, `equals()` y `hashCode()` basados en los valores de los campos. Estos métodos nos permiten trabajar con los datos de forma segura y consistente sin necesidad de código adicional. Por lo tanto, las respuestas correctas son B, C, D y G.

28._ A, B y D

Las clases Camel, EatsGrass y Eagle no compilan porque violan reglas fundamentales de Java. Camel tiene un método sin cuerpo que no está marcado como abstracto. EatsGrass declara un método abstracto como `private`, lo cual no es válido en interfaces. Eagle contiene un método abstracto en una clase concreta, lo cual requiere que la clase sea abstracta.

29._ F

El programa no compila debido a tres errores. Las líneas 5 y 12 son inválidas porque utilizan `this()` de forma incorrecta, ya que `this()` solo puede usarse para invocar constructores y no como referencia a instancia. Además, la línea 15 contiene un casteo inválido, ya que `Orangutan` no tiene relación de herencia con `Primate`. Por lo tanto, hay 3 líneas con error

30._ C y E

La clase `EmperorTamarin` no compila porque extiende una clase `sealed` sin declarar explícitamente si es `final`, `sealed` o `non-sealed`. La clase `Friendly` tampoco compila porque incluye en su cláusula `permits` una clase (`Silly`) que no extiende a `Friendly`, lo cual no es válido